

Implementation in Java for simulating simple **Hello** and **Echo** commands over a network using **socket programming**:

Concept Overview:

- **Hello Command** checks if a remote machine is up by receiving a "Hello" response.
- **Echo Command** sends a message and calculates the round-trip time (RTT).

Implementation:

We'll create two Java programs:

1. **Server Program**: Listens for client requests and responds to "HELLO" and "ECHO" commands.
2. **Client Program**: Sends "HELLO" and "ECHO" commands to the server and measures the RTT.

1. Server Program (HelloEchoServer.java):

```
import java.io.*;
import java.net.*;

public class HelloEchoServer {
    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(5555);
        System.out.println("Server is listening on port 5555...");
        while (true) {
            Socket clientSocket = serverSocket.accept();
            System.out.println("Client connected: " + clientSocket.getInetAddress());
            // Handle client in new thread
            new Thread(() -> {
                try {
                    BufferedReader in = new BufferedReader(
                        new InputStreamReader(clientSocket.getInputStream()));
                    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
                    String command;
                    while ((command = in.readLine()) != null) {
                        System.out.println("Received: " + command);
```

```

        if (command.equalsIgnoreCase("HELLO")) {
            out.println("HELLO_RESPONSE");
        } else if (command.startsWith("ECHO ")) {
            String message = command.substring(5);
            out.println("ECHO_RESPONSE " + message);
        } else if (command.equalsIgnoreCase("EXIT")) {
            break;
        } else {
            out.println("UNKNOWN_COMMAND");
        }
    }
    clientSocket.close();
    System.out.println("Client disconnected.");
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
}).start();
}
}
}

```

2. Client Program (HelloEchoClient.java):

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class HelloEchoClient {

    public static void main(String[] args) throws IOException {

        Socket socket = new Socket("localhost", 5555);
        System.out.println("Connected to server at localhost:5555");
        BufferedReader in = new BufferedReader(

```

```

        new InputStreamReader(socket.getInputStream()));
    PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
    Scanner scanner = new Scanner(System.in);
    String input;
    System.out.println("Commands:\n1. HELLO\n2. ECHO <message>\n3. EXIT");
    while (true) {
        System.out.print("Enter command: ");
        input = scanner.nextLine();
        if (input.equalsIgnoreCase("EXIT")) {
            out.println("EXIT");
            break;
        }
        if (input.equalsIgnoreCase("HELLO")) {
            out.println("HELLO");
            String response = in.readLine();
            if (response != null && response.equals("HELLO_RESPONSE"))
                System.out.println("Server is alive!");
            else
                System.out.println("Unexpected response: " + response);
        } else if (input.startsWith("ECHO ")) {
            long startTime = System.currentTimeMillis();
            out.println(input);
            String response = in.readLine();
            long endTime = System.currentTimeMillis();
            if (response != null && response.startsWith("ECHO_RESPONSE")) {
                System.out.println("Server echoed: " + response.substring(14));
                System.out.println("Round-trip time: " + (endTime - startTime) + " ms");
            } else {
                System.out.println("Unexpected response: " + response);
            }
        } else {

```

```
        System.out.println("Invalid command!");
    }
}
socket.close();
scanner.close();
System.out.println("Disconnected from server.");
}
}
```

Example commands explanation:

- HELLO: Checks if server responds correctly.
- ECHO: Sends a message and measures round-trip time.
- EXIT: Ends the connection.

How to Run:

Step 1: Compile both files

```
javac HelloEchoServer.java
javac HelloEchoClient.java
```

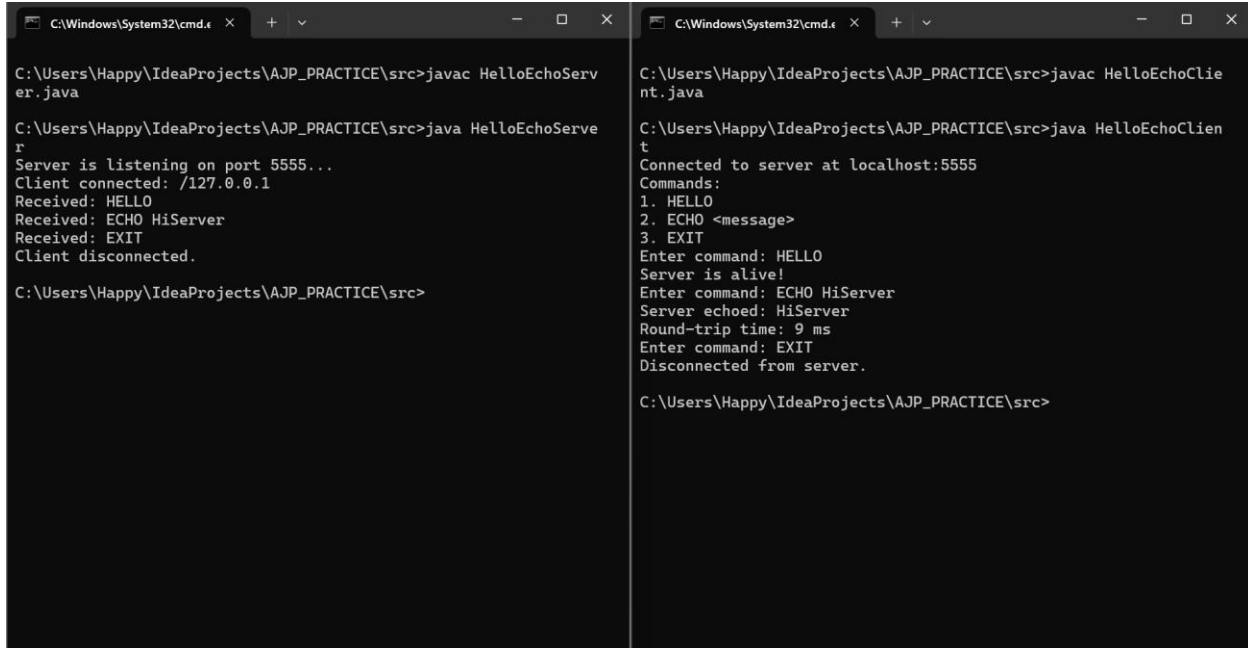
Step 2: Run the server (First terminal)

```
java HelloEchoServer
```

Step 3: Run the client (Second terminal)

```
java HelloEchoClient
```

Output:



```
C:\Windows\System32\cmd.exe x + v
C:\Users\Happy\IdeaProjects\AJP_PRACTICE\src>javac HelloEchoServer.java

C:\Users\Happy\IdeaProjects\AJP_PRACTICE\src>java HelloEchoServer
Server is listening on port 5555...
Client connected: /127.0.0.1
Received: HELLO
Received: ECHO HiServer
Received: EXIT
Client disconnected.

C:\Users\Happy\IdeaProjects\AJP_PRACTICE\src>
```

```
C:\Windows\System32\cmd.exe x + v
C:\Users\Happy\IdeaProjects\AJP_PRACTICE\src>javac HelloEchoClient.java

C:\Users\Happy\IdeaProjects\AJP_PRACTICE\src>java HelloEchoClient
Connected to server at localhost:5555
Commands:
1. HELLO
2. ECHO <message>
3. EXIT
Enter command: HELLO
Server is alive!
Enter command: ECHO HiServer
Server echoed: HiServer
Round-trip time: 9 ms
Enter command: EXIT
Disconnected from server.

C:\Users\Happy\IdeaProjects\AJP_PRACTICE\src>
```